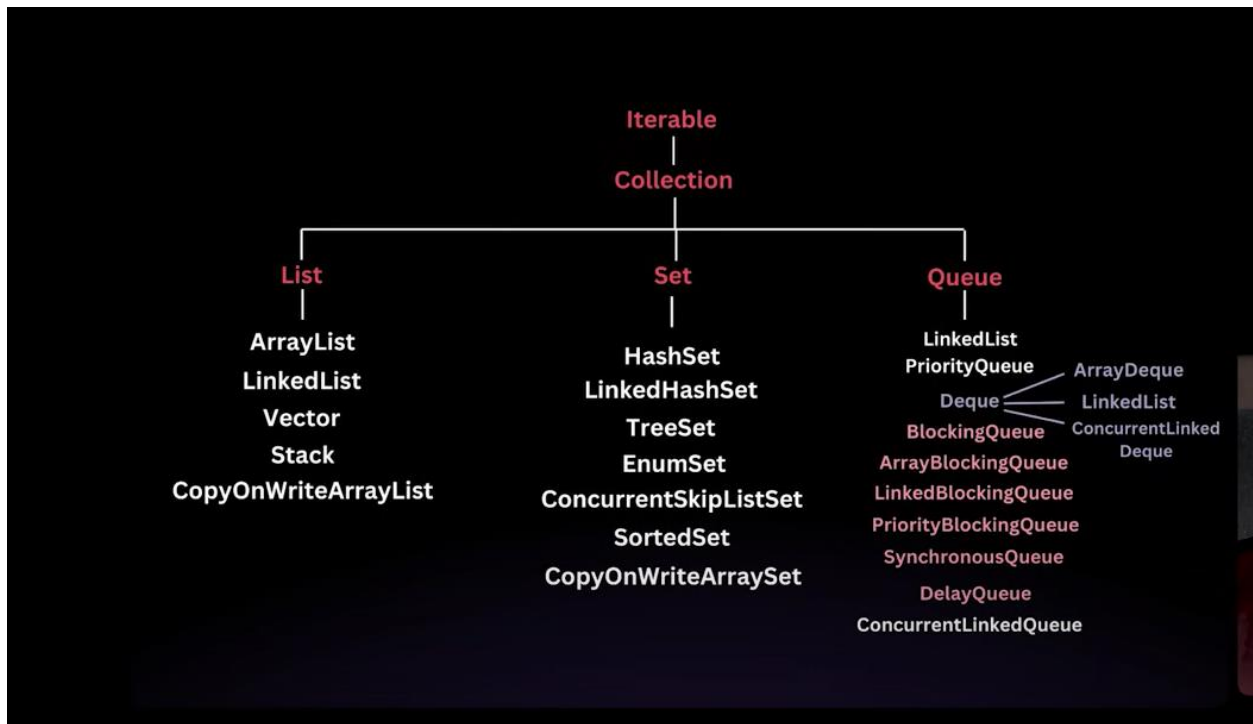




◆ 1. ArrayList

■ Definition:

ArrayList is a **resizable array** implementation of the List interface.

⚙ Internal Working:

- Uses a **dynamic array** to store elements.
- Default initial capacity: **10**
- Array size grows automatically when capacity is exceeded.

Growth Factor:

- When full, it creates a **new array of 1.5 times (50%)** the current size:
- `newCapacity = oldCapacity + (oldCapacity >> 1);`

Important Methods:

- `add(E e)`
- `get(int index)`
- `set(int index, E element)`
- `remove(int index)`
- `contains(Object o)`
- `size()`
- `clear()`

Time Complexity:

Operation	Time Complexity
<code>add()</code> (end)	$O(1)$ amortized
<code>get()</code>	$O(1)$
<code>remove(index)</code>	$O(n)$
<code>contains()</code>	$O(n)$
<code>add(index)</code>	$O(n)$

◆ 2. LinkedList

Definition:

LinkedList is a **doubly linked list** implementation of the List and Deque interfaces.

⚙️ Internal Working:

- Each node stores data and pointers to the **next and previous nodes**.
- No resizing needed as it does not use an array.

📊 Growth Factor:

- Not applicable — it uses nodes, not arrays.

🧰 Important Methods:

- add(E e)
- addFirst(E e)
- addLast(E e)
- get(int index)
- remove(int index)
- removeFirst()
- removeLast()
- size()

🕒 Time Complexity:

Operation	Time Complexity
-----------	-----------------

addFirst/Last()	$O(1)$
-----------------	--------

get(index)	$O(n)$
------------	--------

remove(index)	$O(n)$
---------------	--------

contains()	$O(n)$
------------	--------

◆ 3. Vector

📘 Definition:

Vector is a **synchronized**, dynamic array-based list.

⚙️ Internal Working:

- Similar to ArrayList but **thread-safe** (methods are synchronized).
- Used in legacy code.

Growth Factor:

- Grows by **doubling the size (2x)** when capacity is reached.

Important Methods:

- add(E e)
- get(int index)
- remove(int index)
- size()
- capacity()
- elementAt(int index)

Time Complexity:

Operation	Time Complexity
add()	O(1) amortized
get()	O(1)
remove(index)	O(n)
contains()	O(n)

◆ 4. Stack

Definition:

Stack is a **LIFO (Last In First Out)** subclass of Vector.

Internal Working:

- Inherits from Vector.
- Methods operate at the top of the stack.

Important Methods:

- `push(E item)` – Add item to top
- `pop()` – Remove and return top
- `peek()` – View top without removing
- `isEmpty()`

Time Complexity:

Operation Time Complexity

`push()` $O(1)$

`pop()` $O(1)$

`peek()` $O(1)$

◆ **5. CopyOnWriteArrayList**

Definition:

A thread-safe variant of `ArrayList` where **modification operations (add, set, remove)** result in a **new copy of the array**.

Internal Working:

- Used for **concurrent read-heavy use cases**.
- Every update creates a **new copy** of the array.
- Provides **fail-safe iterators** (no `ConcurrentModificationException`).

Growth Factor:

- Similar to `ArrayList`, but copies on each modification.

Important Methods:

- `add(E e)`
- `remove(Object o)`
- `get(int index)`
- `iterator()` – snapshot of the array

Time Complexity:

Operation Time Complexity

add() $O(n)$ (due to copy)

get() $O(1)$

remove() $O(n)$

contains() $O(n)$

Summary Table

Feature / List Type	ArrayList	LinkedList	Vector	Stack	CopyOnWriteArrayList
Backed By	Array	LinkedList	Array	Array	Array (copy on write)
Thread Safe	✗	✗	✓	✓	✓
Random Access	✓	✗	✓	✓	✓
Insert/Delete Middle	Slow	Fast	Slow	Slow	Very Slow
Read Performance	Fast	Medium	Fast	Fast	Fast
Write Performance	Fast	Medium	Medium	Medium	Slow (copy)
Growth Factor	1.5x	N/A	2x	2x	1.5x + copy

Comparator -

<https://www.geeksforgeeks.org/java-comparator-interface/>

Map-

A HashMap is part of Java's **Collection Framework**, and it's used to **store key-value pairs**. It's one of the most commonly used classes in Java when you need to associate a key with a value and retrieve it quickly.

✅ Basic Concept

- A HashMap stores **data in key-value pairs**.
- Each **key is unique**, but values can be **duplicated**.
- It uses a **hashing mechanism** to store and retrieve values efficiently.

🔍 Key Methods

Method	Description
put(key, value)	Adds a key-value pair
get(key)	Returns the value associated with the key
remove(key)	Removes the key-value pair
containsKey(key)	Checks if a key exists
containsValue(val)	Checks if a value exists
keySet()	Returns a set of all keys
values()	Returns a collection of all values
entrySet()	Returns a set of key-value pairs

Points to Remember

- It is **not synchronized** → use `Collections.synchronizedMap()` or `ConcurrentHashMap` for thread-safe operations.
- **null keys and values**: It allows **one null key** and **multiple null values**.

Here's a **detailed explanation of how HashMap works internally in Java**, including its **structure, hashing mechanism, collision handling, resizing**, and more.

🏠 1. Internal Structure

A HashMap in Java is essentially a **bucket-based structure**. Internally, it is built like this:

```
transient Node<K,V>[] table;
```

Each bucket is a **linked list** (or a red-black tree for performance when needed).

Each entry in the bucket is of type:

```
static class Node<K,V> implements Map.Entry<K,V> {  
    final int hash;  
    final K key;  
    V value;  
    Node<K,V> next;  
}
```

2. Insertion (put() method)

```
map.put("Apple", 10);
```

Steps:

1. Compute hash:

- `hash = key.hashCode()` (e.g., `"Apple".hashCode()`)
- Enhanced by `hash(hashCode)` to reduce collisions:
- ```
static final int hash(Object key) {
```
- ```
    int h;
```
- ```
 return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
```
- ```
}
```

2. Calculate index in table:

- `index = hash % table.length` (or, internally, `hash & (n - 1)` for performance).

3. Insert in the bucket:

- If the bucket is empty: insert node.
- If the bucket already has entries (collision): traverse using `.equals()` to check for same key:

- If key exists → replace value.
- Else → add to the end of the list.

⚠ 3. Collision Handling

When multiple keys hash to the same index, it's called a **collision**.

Java handles it using:

- **LinkedList** (for earlier versions or fewer collisions)
- **Red-Black Tree** (since Java 8, if the number of items in a bucket > TREEIFY_THRESHOLD, usually 8)

🔥 4. Threshold and Load Factor

- **Load Factor**: Default is 0.75.
- When $\text{size} \geq \text{capacity} * \text{load factor}$, the HashMap **resizes** (doubles capacity) to avoid too many collisions.

For example:

- If initial capacity is 16 and load factor is 0.75,
- Resize occurs when $16 \times 0.75 = 12$ entries are added.

🔄 5. Resizing (Rehashing)

When resizing:

- A new array with **double size** is created.
- **All entries are rehashed** and moved to the new array.
- It's **expensive** ($O(n)$), so it's done sparingly.

✅ 6. Retrieval (get() method)

`map.get("Apple");`

1. Compute hash from key.
 2. Compute index in the table.
 3. Traverse the bucket (LinkedList or Tree) to find a node with equals() matching the key.
 4. Return the value.
-

7. Null Key and Values

- HashMap allows **one null key** and **multiple null values**.
 - The null key always goes in **bucket 0**.
-

Example Internals

```
HashMap<String, Integer> map = new HashMap<>();
```

```
map.put("Apple", 1);
```

```
map.put("Banana", 2);
```

```
map.put("Orange", 3);
```

Internally, it may look like:

Index Entries

- | | |
|---|-------------------------------|
| 0 | null |
| 1 | Node(hash, "Apple", 1, null) |
| 2 | Node(hash, "Banana", 2, null) |
| 3 | Node(hash, "Orange", 3, null) |

If "Banana" and "Orange" have the same hash index, you get a linked list or a tree in that bucket.

Summary

Feature	Description
Hashing	Used to calculate index from key
Collision	Handled with LinkedList or Tree
Load Factor	Triggers resizing when capacity exceeded
Null Handling	One null key allowed
Resizing	Expensive operation involving rehashing
Time Complexity	Average $O(1)$, Worst-case $O(n)$ for get/put

LinkedHashMap — what you need to remember, how it works internally, and how it's useful for **LRU caching**.

LinkedHashMap: Key Points to Remember

Feature	Description
Inherits From	HashMap
Maintains Insertion Order	✅ Yes – elements are returned in the order they were inserted
Access Order Option	✅ Can be set to maintain access order (important for LRU cache)
Allows Null Key/Value	✅ Yes – one null key, multiple null values (like HashMap)
Not Thread-Safe	❌ Use <code>Collections.synchronizedMap()</code> or <code>ConcurrentHashMap</code> if needed
Time Complexity	Same as HashMap – $O(1)$ for get/put

Internal Working of LinkedHashMap

Structure

- LinkedHashMap extends HashMap and adds a **doubly linked list** to maintain order.
- Internally uses a class like:

```
static class Entry<K,V> extends HashMap.Node<K,V> {
    Entry<K,V> before, after;
}
```

Order Modes

1. Insertion Order (default):

- Keeps keys in the order they were inserted.

2. Access Order:

- If accessOrder=true, moves accessed keys (via get(), put()) to the end of the list.
- Useful for LRU cache.

Constructor to set access order:

```
LinkedHashMap(int capacity, float loadFactor, boolean accessOrder)
```

Use Case: LRU (Least Recently Used) Caching

LRU cache removes the **least recently used** entry when capacity is full.

Implementation using LinkedHashMap

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // true = access order
        this.capacity = capacity;
    }
```

```
@Override
protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
    return size() > capacity;
}
}
```

✅ Usage:

```
public class Main {
    public static void main(String[] args) {
        LRUCache<Integer, String> cache = new LRUCache<>(3);

        cache.put(1, "One");
        cache.put(2, "Two");
        cache.put(3, "Three");
        cache.get(1); // Access 1
        cache.put(4, "Four"); // 2 is least recently used → removed

        System.out.println(cache.keySet()); // Output: [3, 1, 4]
    }
}
```

Here's a complete and **clear explanation of WeakHashMap** — when to use it, how it works, and how **weak vs. strong references** tie into **Java Garbage Collection**.

🌿 WeakHashMap in Java – Overview

🔑 What is WeakHashMap?

- A special Map implementation where **keys are stored using *weak references***.
- If a key is no longer **strongly reachable**, it **can be garbage collected**.
- Once the key is GC'd, the entry is **automatically removed** from the map.

```
Map<Key, Value> map = new WeakHashMap<>();
```

When to Use WeakHashMap

Use Case	Example
Caching with automatic cleanup	e.g., caching metadata like image previews
Avoiding memory leaks for large maps where keys may become unreachable	listeners, temporary computations
When you don't want to manage memory cleanup manually	Java cleans up when the key becomes unreachable

What is a Strong Reference?

A normal reference in Java. As long as a strong reference exists, the object **won't be garbage collected**.

```
MyObject obj = new MyObject(); // Strong reference
```

As long as obj is in scope, it's **protected from GC**.

What is a Weak Reference?

A reference that **doesn't prevent** its object from being collected by the GC.

In Java, you can explicitly create one using:

```
WeakReference<MyObject> weakRef = new WeakReference<>(new MyObject());
```

If no strong reference exists, the object can be GC'd, even if a weak reference exists.

🧠 How WeakHashMap Uses Weak References

- It wraps **keys** in WeakReference internally.
- The Value is still **strongly referenced**.
- When the key is no longer referenced elsewhere, the GC **cleans up the entry**.

```
Map<Object, String> map = new WeakHashMap<>();
```

```
Object key = new Object();
```

```
map.put(key, "value");
```

```
key = null; // Remove strong reference
```

```
System.gc(); // Suggest GC
```

```
// The map entry will likely be removed
```

💡 Garbage Collection & WeakHashMap

💬 Why GC removes entries?

- Because the **key object is weakly referenced** and not strongly reachable.
- GC identifies it as **eligible for collection**, removes it, and cleans up the map automatically.

📊 Comparison Table

Feature	HashMap	WeakHashMap
Key reference type	Strong	Weak
Entry removed on GC?	❌ No	✅ Yes (if key GC'd)
Suitable for caching?	⚠️ Careful manual	✅ Automatic cleanup

Feature	HashMap	WeakHashMap
Thread-safe?	✗ No	✗ No

👁️ Real Example: Memory-sensitive Caching

```
import java.util.Map;

import java.util.WeakHashMap;

public class WeakMapDemo {

    public static void main(String[] args) throws InterruptedException {

        Map<Object, String> map = new WeakHashMap<>();

        Object key1 = new Object();
        Object key2 = new Object();

        map.put(key1, "Value1");
        map.put(key2, "Value2");

        System.out.println("Before GC: " + map);

        key1 = null; // make key1 unreachable

        System.gc();

        Thread.sleep(1000); // give GC some time

        System.out.println("After GC: " + map);
    }
}
```



```
}  
}
```

Output might be:

Before GC: {java.lang.Object@1a -> Value1, java.lang.Object@2b -> Value2}

After GC: {java.lang.Object@2b -> Value2}

What is IdentityHashMap?

IdentityHashMap is a **Map implementation** that uses **reference equality (==)** instead of **object equality (.equals())** for comparing keys.

Example:

```
Map<String, String> map = new IdentityHashMap<>();
```

```
map.put(new String("a"), "value1");
```

```
map.put(new String("a"), "value2");
```

```
System.out.println(map.size()); // Output: 2 (because two different objects)
```

Even though the strings have the same value, they are **different objects**, so both entries are kept.

Key Characteristics

Feature	Value
Uses .equals()?	✗ No
Uses ==?	✓ Yes – reference equality
Allows null keys?	✓ Yes
Allows duplicates?	✓ For <i>equal content but different objects</i>
Order maintained?	✗ No

Feature	Value
Thread-safe?	✗ No
Backed by	Array-based open-addressing hash table

⚙ Internal Working

- Internally uses **open-addressed arrays** instead of linked nodes.
- Keys are compared using `==`, **not `.equals()`**.
- Uses `System.identityHashCode(key)` instead of `key.hashCode()`.

`System.identityHashCode(key)` // returns the default object hash, like `Object::hashCode`

So:

```
String s1 = new String("abc");
```

```
String s2 = new String("abc");
```

```
System.out.println(s1 == s2); // false
```

```
System.out.println(s1.equals(s2)); // true
```

```
map.put(s1, "one");
```

```
map.put(s2, "two"); // Both entries stay in IdentityHashMap
```

📌 When to Use IdentityHashMap

Use Case	Why
Object identity tracking	When you care about exact object instances , not logical equality
Reference equality needed	For example, in serialization, intern pool lookups, dependency graphs

Use Case	Why
Performance-sensitive scenarios	Slightly faster due to no chaining or equals() call overhead

⚠ Don't Use When...

Case	Why
You want logical equality (equals())	Use HashMap instead
You override .equals() in your key class	IdentityHashMap ignores it
You expect standard Map behavior	It's a special-purpose tool only

📄 Quick Code Demo

```
import java.util.*;

public class IdentityMapDemo {
    public static void main(String[] args) {
        Map<String, String> map = new IdentityHashMap<>();

        String key1 = new String("hello");
        String key2 = new String("hello");

        map.put(key1, "Value1");
        map.put(key2, "Value2");

        System.out.println(map);    // {hello=Value1, hello=Value2}
        System.out.println(map.size()); // 2
    }
}
```

```
}  
  
}
```

IdentityHashMap vs HashMap

Feature	HashMap	IdentityHashMap
Key comparison	.equals()	== (reference equality)
Hash function	key.hashCode()	System.identityHashCode(key)
Allows duplicate keys?	 Based on .equals()	 If different object refs
Use Case	General-purpose	Identity-sensitive scenarios

What is TreeMap?

- A **Map** that stores keys in **sorted order** (according to their natural ordering or a custom Comparator).
- Implements both SortedMap and NavigableMap.
- Based on a **Red-Black Tree**, a type of self-balancing binary search tree.

```
Map<Integer, String> map = new TreeMap<>();
```

```
map.put(10, "A");
```

```
map.put(5, "B");
```

```
map.put(20, "C");
```

```
System.out.println(map); // Output: {5=B, 10=A, 20=C}
```

Key Characteristics of TreeMap

Feature	Description
Ordering	Sorted by natural order or Comparator

Feature	Description
Null Keys	✗ Not allowed (throws NullPointerException)
Null Values	✓ Allowed
Time Complexity	$O(\log n)$ for get, put, remove
Thread-safe	✗ No
Implements	NavigableMap, SortedMap, Map

TreeMap Implements:

◆ **SortedMap (interface)**

- Guarantees keys are sorted.
- Common methods:
 - firstKey()
 - lastKey()
 - subMap(fromKey, toKey)
 - headMap(toKey)
 - tailMap(fromKey)

◆ **NavigableMap (extends SortedMap)**

- More powerful navigation:
 - lowerKey(k) – greatest key $< k$
 - floorKey(k) – greatest key $\leq k$
 - ceilingKey(k) – least key $\geq k$
 - higherKey(k) – least key $> k$
 - pollFirstEntry() / pollLastEntry() – remove and return
-

Internal Working of TreeMap

- Backed by a **Red-Black Tree**:
 - A self-balancing binary search tree.
 - Ensures the tree height is always $O(\log n)$ for fast lookup and insertion.
 - TreeMap entries are stored as **nodes in the Red-Black Tree**.
- Keys must be **comparable** (Comparable interface) or a **custom Comparator** must be provided.

Red-Black Tree Properties:

1. Each node is either red or black.
2. Root is always black.
3. Red nodes cannot have red children.
4. Every path from a node to descendant null nodes has the same number of black nodes.

These rules guarantee **logarithmic height**, making all operations efficient.

Real Use Cases of TreeMap

Use Case	Why TreeMap?
Sorted leaderboards	Automatically maintains ranking
Scheduling events (by time)	Keys (timestamps) are naturally ordered
Range queries	With subMap, headMap, etc.
Implementing priority tasks	Sorted priority can be efficiently handled

Code Examples

Natural Ordering (default):

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(3, "C");
map.put(1, "A");
```

```
map.put(2, "B");
```

```
System.out.println(map); // {1=A, 2=B, 3=C}
```

Custom Comparator:

```
TreeMap<String, Integer> map = new TreeMap<>(Comparator.reverseOrder());
```

```
map.put("apple", 10);
```

```
map.put("banana", 5);
```

```
System.out.println(map); // {banana=5, apple=10}
```

NavigableMap methods:

```
TreeMap<Integer, String> map = new TreeMap<>();
```

```
map.put(10, "A");
```

```
map.put(20, "B");
```

```
map.put(30, "C");
```

```
System.out.println(map.lowerKey(25)); // 20
```

```
System.out.println(map.floorKey(20)); // 20
```

```
System.out.println(map.ceilingKey(25)); // 30
```

```
System.out.println(map.higherKey(10)); // 20
```

TreeMap vs HashMap vs LinkedHashMap

Feature	HashMap	LinkedHashMap	TreeMap
Order	 Unordered	 Insertion/Access Order	 Sorted Order
Null key	 1 allowed	 1 allowed	 Not allowed
Performance	O(1)	O(1)	O(log n)
Backed by	Hash table	Hash table + linked list	Red-Black Tree

Feature	HashMap	LinkedHashMap	TreeMap
Best for	Fast lookups	Caching	Range queries, sorted data

✅ Summary

- TreeMap is a **Red-Black Tree-based Map**.
- Maintains **sorted order** of keys.
- Implements **SortedMap** and **NavigableMap**.
- Use when you need **sorted keys**, **range views**, or **navigation operations**.
- Slightly slower than HashMap but **great for order-sensitive use cases**.

Great notes! Let me help you **polish and expand** the explanation of ConcurrentHashMap and briefly introduce ConcurrentSkipListMap for clarity and interview readiness:

-
-  **ConcurrentHashMap (CHM) – Thread-Safe HashMap**
 -  **Key Points:**
 - Part of java.util.concurrent
 - Allows **concurrent access** by multiple threads **without locking the whole map**
 - Null keys and values are **not allowed**
-
-  **Internal Working**
 -  **Java 7 – Segmented Locking**
 - Internally divided into **16 segments** (by default)
 - Each segment is a smaller HashMap with its own lock
 - Read operations **don't need locks**, unless a write is happening in the same segment

- Write operations **lock only the relevant segment**
- Pros: Better concurrency than `Collections.synchronizedMap`
- Cons: Still uses locks → performance overhead
- `ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();`

- 🧑 **Java 8+ – Lock-Free with CAS**

- Segments are **removed**: uses a **bucket array (Node[])**, similar to `HashMap`
- Core mechanism: **CAS (Compare-And-Swap)**
- CAS = Optimistic locking:
- Read latest value → compute new value → if value hasn't changed → **update**
- If value changed → retry
- **synchronized** is used only:
- During **resizing** (e.g., doubling table size)
- On **high collision** in a bin (converts bin to a `TreeBin`)
- ✅ Lock-free for reads and most writes
- ✅ Efficient even with high concurrency

- 💻 **Example – CAS in `putIfAbsent`:**

- `map.putIfAbsent("key", 1);` // Insert if "key" not already present
- Internally:
- Checks if "key" exists
- If not → uses CAS to insert
- Otherwise → doesn't overwrite

- 🚀 **When to Use `ConcurrentHashMap`**

- Multi-threaded environments where high **read and write performance** is needed
- Alternative to `Hashtable` or `Collections.synchronizedMap`

- Avoids ConcurrentModificationException during iteration
- Used in caches, thread-safe registries, etc.

Here's a detailed **interview-ready explanation** of ConcurrentSkipListMap — covering its **internal working, structure, concurrency handling**, and **when to use it**.

🧩 What is ConcurrentSkipListMap?

- A **thread-safe, sorted, non-blocking** implementation of NavigableMap.
- Maintains **sorted key order** (natural or custom comparator).
- Internally uses a **Skip List**, not a tree.
- Part of java.util.concurrent.

```
ConcurrentSkipListMap<Integer, String> map = new ConcurrentSkipListMap<>();  
map.put(5, "A");  
map.put(2, "B");  
map.put(10, "C");  
System.out.println(map); // Output: {2=B, 5=A, 10=C}
```

🔍 Key Characteristics

Feature	Description
Sorted	✅ Yes (natural or comparator order)
Thread-safe	✅ Yes
Null keys/values	❌ Not allowed

Feature	Description
Time complexity	$O(\log n)$ for get, put, remove
Backed by	Skip List (not Red-Black Tree like TreeMap)
Concurrent behavior	Non-blocking reads & fine-grained updates

⚙️ Internal Working – Skip List

🧠 What is a Skip List?

- A **multi-level linked list** that allows fast traversal.
- Each level is a **sorted list**, and higher levels "**skip**" over more elements.
- Nodes exist in **multiple levels**, promoting faster lookups.

Level 3: A -----> G

Level 2: A --> C -----> G

Level 1: A -> B -> C -> D -> E -> F -> G

- Searching for "E":
 - Start at top
 - Move right until you overshoot
 - Drop down one level
 - Repeat

➡️ Average time complexity: $O(\log n)$

🔒 Concurrency

- Operations use **non-blocking algorithms**:
 - Based on **CAS (Compare-And-Swap)** and **volatile** fields
 - Lock-free reads, fine-grained locks or retries for writes
 - **No global locking** = highly concurrent
-

How Operations Work

✓ put(K key, V value)

- Traverse the skip list levels to find insert position
- Create a new node with random levels
- Use **CAS** to insert safely without blocking

✓ get(Object key)

- Start from top level and move down/right
- No locking; purely reads via volatile pointers

✓ remove(Object key)

- Find node using traversal
- Use CAS to mark as deleted
- Physically remove later (lazy deletion)

Common Methods

Method	Description
put(K, V)	Adds a key-value pair
get(K)	Returns value for the key
remove(K)	Removes key
firstKey() / lastKey()	Smallest / largest key
ceilingKey(K)	Smallest key $\geq$ K
floorKey(K)	Largest key $\leq$ K
subMap(from, to)	Range query

Why Use ConcurrentSkipListMap?

Situation	Why it's suitable
Need sorted keys	Automatically maintains order
Range queries in concurrent env	subMap(), headMap() are safe & fast
High read/write concurrency	Non-blocking & lock-free where possible
Alternative to TreeMap in multi-threading	TreeMap is not thread-safe

Comparison with Other Maps

Feature	HashMap	ConcurrentHashMap	TreeMap	ConcurrentSkipListMap
Thread-safe	✗	✓ Yes	✗	✓ Yes
Sorted	✗	✗	✓ Yes	✓ Yes
Backing structure	Hash table	Hash table (no lock)	Red-Black Tree	Skip List
Null key/value	✓ / ✓	✗ / ✗	✗ / ✓	✗ / ✗
Best for	Simple fast	Concurrent + fast	Sorted (single threaded)	Sorted + concurrent

Here's a complete breakdown of **EnumMap** and **Immutable Maps** in Java — internal working, when to use, and key characteristics, especially from an **interview perspective**.

⚙️ EnumMap<K extends Enum<K>, V>

✅ What It Is:

- A **Map** optimized specifically for enum keys
- Part of java.util
- Backed internally by an **array**, not a hash table
- Very **fast** and **memory efficient**

```
enum Day { MON, TUE, WED }
```

```
EnumMap<Day, String> schedule = new EnumMap<>(Day.class);
```

```
schedule.put(Day.MON, "Gym");
```

```
schedule.put(Day.TUE, "Work");
```

⚙️ Internal Working

- Internally uses an **array indexed by ordinal()** of enum:
 - For example: `Day.MON.ordinal() == 0`
- Since enum constants are **known and finite**, index-based access is **constant time O(1)**
- No hashing or collisions

Example:

```
schedule.put(Day.TUE, "Work");
```

→ internally:

```
array[Day.TUE.ordinal()] = "Work";
```

⚠ Constraints

Aspect	Details
Key type	Must be an enum
Null keys	❌ Not allowed
Null values	✅ Allowed
Thread safety	❌ Not thread-safe (wrap with <code>Collections.synchronizedMap</code>)

🔧 When to Use EnumMap

- When you use enum as keys
 - Performance-critical applications
 - Predictable and limited number of keys
-

📦 Immutable Map

✅ What It Is:

- A map that **cannot be modified** once created
- Introduced in **Java 9**: `Map.of()` and `Map.ofEntries()`
- Also available in **Guava**: `ImmutableMap.of()`

```
Map<String, String> map = Map.of("a", "apple", "b", "banana");
```

```
// map.put("c", "cherry"); ❌ UnsupportedOperationException
```

🧠 Why Use Immutable Map

- **Thread-safe** without synchronization
- **Safe by design**: avoids bugs from accidental modifications
- Used for:
 - Configurations

- Constants
- Lookup tables

Bonus Tip for Interviews:

- **ImmutableMap** (like in Guava or Java 9+) is ideal for constants:

```
public static final Map<String, String> ERROR_CODES = Map.of(
    "404", "Not Found",
    "500", "Internal Server Error"
);
```

- **EnumMap** is best when keys are enums and **performance is important**

This is **crucial to understand** when using custom objects as keys in HashMap, HashSet, or any other hash-based collection.

Why Override equals() and hashCode()?

When you store a custom object as a **key in a HashMap**, Java uses:

- hashCode() to find the bucket (i.e., index in internal array)
- equals() to compare keys in the same bucket

If you **don't override** them properly, your map will not behave as expected.

Internal Working of HashMap (Simplified):

1. **Put (map.put(key, value)):**

- Calls key.hashCode() → determines bucket index
- If other keys are in the same bucket, uses equals() to check for key equality

2. **Get (map.get(key)):**

- Same process: hashCode() to locate bucket → equals() to find exact key

🔔 What Happens If You Don't Override?

Example:

java

CopyEdit

```
class Person {  
    String name;  
    Person(String name) { this.name = name; }  
}
```

```
Map<Person, String> map = new HashMap<>();
```

```
map.put(new Person("Alice"), "Engineer");
```

```
// Try to retrieve
```

```
System.out.println(map.get(new Person("Alice"))); // ❌ null
```

Even though the new Person("Alice") *looks* the same, it's not equal by default because:

- Default equals() checks **object identity** (==)
- Default hashCode() uses **memory address hash**

So HashMap thinks they are **different keys**!

✅ Correct Way:

java

CopyEdit

```
class Person {  
    String name;  
    Person(String name) { this.name = name; }
```

@Override

```
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (!(o instanceof Person)) return false;  
    Person p = (Person) o;  
    return name.equals(p.name);  
}
```

@Override

```
public int hashCode() {  
    return name.hashCode(); // or use Objects.hash(name)  
}  
}
```

Now:

java

CopyEdit

```
map.put(new Person("Alice"), "Engineer");
```

```
System.out.println(map.get(new Person("Alice"))); //  "Engineer"
```

Summary (Interview Ready)

Concept	Why It Matters
---------	----------------

hashCode()	Used to compute bucket index in HashMap
------------	---

equals()	Used to compare keys in the same bucket
----------	---

Not overriding Causes duplicate keys or retrieval failures

Concept Why It Matters

Override both Ensures proper behavior in hash-based collections

💡 Golden Rule:

Whenever you override equals(), always override hashCode() too — and vice versa — to maintain the contract.

Here's a clear and concise explanation of various **Set implementations** in Java — especially useful for interviews and production usage.

🔗 What is a Set?

- A **collection** that **doesn't allow duplicate elements**
 - Defined in the java.util.Set interface
 - Commonly used to store unique items
-

📚 Types of Sets in Java

Set Type	Backed By / Internal Working	Ordering	Thread Safe	Null Allowed?
HashSet	HashMap	❌ No order	❌ No	✅ 1 null
LinkedHashSet	HashMap + Linked List	✅ Insertion order	❌ No	✅ 1 null
TreeSet	Red-Black Tree (NavigableSet)	✅ Sorted	❌ No	❌ No nulls
ConcurrentSkipListSet	Skip List (ConcurrentNavigableSet)	✅ Sorted	✅ Yes	❌ No nulls

Set Type	Backed By / Internal Working	Ordering	Thread Safe	Null Allowed?
EnumSet	Bit Vector / Array (for Enums only)	✅ Natural Enum	❌ No	❌ No nulls
CopyOnWriteArraySet	Backed by CopyOnWriteArrayList	✅ Insertion order	✅ Yes (slow writes)	✅ 1 null

✿ 1. HashSet

◆ Characteristics:

- Backed by a HashMap
- Elements are **unordered**
- Allows **1 null**
- Fast performance: $O(1)$ for add, remove, contains

```
Set<String> set = new HashSet<>();
```

```
set.add("A"); set.add("B"); set.add("A"); // Duplicates ignored
```

🔧 Use Case:

- When order **doesn't matter** and **speed** is important

🧱 2. LinkedHashSet

◆ Characteristics:

- Maintains **insertion order**
- Slightly **slower** than HashSet
- Internally uses a **linked list** on top of HashMap

```
Set<String> set = new LinkedHashSet<>();
```

```
set.add("B"); set.add("A"); // Iteration: B, A
```

Use Case:

- When you want **uniqueness + predictable iteration order**
-

3. TreeSet

◆ Characteristics:

- Backed by a **Red-Black Tree**
- Sorted according to **natural order** or **custom Comparator**
- Implements `NavigableSet`, `SortedSet`

```
Set<Integer> set = new TreeSet<>();
```

```
set.add(3); set.add(1); set.add(2); // Sorted: 1, 2, 3
```

 **No nulls allowed (throws `NullPointerException`)**

Use Case:

- When you need **sorted elements**
-

4. ConcurrentSkipListSet

◆ Characteristics:

- Thread-safe & lock-free
- Uses **skip list** data structure
- Sorted set (like `TreeSet`) but **concurrent**

```
Set<String> set = new ConcurrentSkipListSet<>();
```

```
set.add("Apple"); set.add("Banana");
```

Use Case:

- Use in **concurrent environments** with **sorted order**
-

5. EnumSet<E extends Enum<E>>

◆ Characteristics:

- Very efficient — internally uses **bit vectors**
- Only works with enum types
- Cannot store null
- All elements must be of the same enum type

```
enum Day { MON, TUE, WED }
```

```
Set<Day> days = EnumSet.of(Day.MON, Day.WED);
```

🔧 Use Case:

- Use when dealing with **fixed sets of enum constants**
 - Much faster than `HashSet<Enum>`
-

🔧 6. CopyOnWriteArraySet

◆ Characteristics:

- Backed by `CopyOnWriteArrayList`
- **Thread-safe** with high **read efficiency**, but **slow writes**
- Every modification creates a new copy of the underlying array

```
Set<String> set = new CopyOnWriteArraySet<>();
```

```
set.add("A"); set.add("B");
```

🔧 Use Case:

- Perfect for **multi-threaded applications** with:
 - **frequent reads**
 - **rare writes**

✅ Queue Overview

A Queue is a **FIFO (First-In-First-Out)** collection.

Java provides:

Interfaces:

- Queue – basic queue
 - Deque – double-ended queue (can insert/remove from both ends)
 - BlockingQueue – thread-safe, used in concurrent scenarios
-

1. Queue Implementations

◆ **LinkedList (also implements Deque)**

- Not thread-safe
- Doubly linked list
- Allows null elements

```
Queue<String> q = new LinkedList<>();
```

```
q.add("A"); q.add("B"); // FIFO
```

Internal Working:

- Each node has prev, next, and data
- Fast insertions/removals from both ends

Use When:

- You want simple **FIFO**, can afford **non-thread-safe**, and allow nulls
-

◆ **PriorityQueue**

- Elements ordered by **natural order** or **Comparator**
- Min-heap internally
- Does **not** allow null
- Not thread-safe

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
pq.add(10); pq.add(5); pq.add(20); // 5 comes out first
```

Internal Working:

- Backed by a **binary heap** (array-based)
- Insertion/removal = **$O(\log n)$**

Use When:

- You need **priority-based processing** (like scheduling, tasks)
-

2. Deque (Double-Ended Queue)

◆ ArrayDeque

- Array-based
- **Faster than LinkedList**
- No thread-safety
- Does not allow null

```
Deque<String> dq = new ArrayDeque<>();
```

```
dq.addFirst("A"); dq.addLast("B");
```

Internal Working:

- Circular array, resizes automatically
- Head and tail pointers managed manually

Use When:

- You want **fast LIFO or FIFO** operations without thread safety
-

◆ LinkedList (also a Deque)

- Backed by a **doubly linked list**
- Slower than ArrayDeque for most deque operations
- Allows null

Use When:

- You need deque behavior **with null allowed** or need **frequent mid-list modifications**
-

◆ ConcurrentLinkedDeque

- **Non-blocking**, thread-safe deque
- Uses **CAS (Compare-And-Swap)**
- Suitable for high-concurrency environments

```
Deque<String> dq = new ConcurrentLinkedDeque<>();
```

```
dq.offerFirst("A");
```

Internal Working:

- Linked nodes with volatile references
- Lock-free using CAS

Use When:

- You need **thread-safe deque** with high throughput, no blocking
-

🔗 3. ConcurrentLinkedQueue

- **Lock-free, thread-safe, non-blocking**
- Based on **linked list** and **CAS**

```
Queue<String> q = new ConcurrentLinkedQueue<>();
```

```
q.add("Task1");
```

Internal Working:

- Uses a linked list of nodes + CAS for thread-safety

Use When:

- In **multi-threaded** applications where **non-blocking queue** is needed
-

🕒 4. BlockingQueue Implementations

These are **thread-safe** and **blocking** queues. Used heavily in **producer-consumer** patterns.

◆ **ArrayBlockingQueue**

- Bounded
- Array-based, uses **lock + condition variables**
- FIFO order
- Doesn't allow null

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(100);
```

Use When:

- You want a **fixed-size queue** with blocking behavior
-

◆ **LinkedBlockingQueue**

- Bounded or unbounded
- Linked list based
- Separate locks for put and take → better concurrency

```
BlockingQueue<String> queue = new LinkedBlockingQueue<>();
```

Use When:

- You want **better performance** under **high concurrency**
-

◆ **PriorityBlockingQueue**

- Unbounded
- Elements sorted by **priority**
- Thread-safe and **non-FIFO**

```
BlockingQueue<Integer> pq = new PriorityBlockingQueue<>();
```

Use When:

- You want a **concurrent** priority queue (like scheduling threads)
-

◆ SynchronousQueue

- **No capacity**
- Each put() waits for a take()
- Zero-length handoff queue

```
BlockingQueue<String> queue = new SynchronousQueue<>();
```

Use When:

- You want a **handoff** mechanism between threads (like thread pools)
-

◆ DelayQueue

- Elements become available **only after a delay**
- Used in caching, retry mechanisms

```
DelayQueue<DelayedTask> queue = new DelayQueue<>();
```

Use When:

- You need **scheduled task execution** (e.g., retries, delayed jobs)
-

Summary Table

Class	Thread Safe	Ordered	Blocking	Special Feature
LinkedList	✗ No	FIFO	✗ No	Doubly-linked
PriorityQueue	✗ No	Sorted	✗ No	Min-heap
ArrayDeque	✗ No	FIFO/LIFO	✗ No	Fast, array-based
ConcurrentLinkedQueue	✓ Yes	FIFO	✗ No	Lock-free, CAS
ConcurrentLinkedDeque	✓ Yes	Both	✗ No	Lock-free deque

Class	Thread Safe	Ordered	Blocking	Special Feature
ArrayBlockingQueue	✓ Yes	FIFO	✓ Yes	Bounded, array
LinkedBlockingQueue	✓ Yes	FIFO	✓ Yes	Linked, better concurrency
PriorityBlockingQueue	✓ Yes	Sorted	✓ Yes	Priority-based
SynchronousQueue	✓ Yes	Direct	✓ Yes	No buffer, 1:1 exchange
DelayQueue	✓ Yes	Delayed	✓ Yes	Delay-based scheduling

✓ Java 8 Functional Programming Concepts

◆ 1. Lambda Expressions

Lambda = Anonymous function

Syntax:

(parameters) -> { body }

Example:

```
Runnable r = () -> System.out.println("Hello from Lambda!");
```

✓ Use Cases:

- Replacing anonymous classes

- Cleaner and more readable code
 - Used in streams, collections, event handling, etc.
-

◆ 2. Functional Interface

- Interface with **only one abstract method**
- Can have default and static methods
- Annotated with `@FunctionalInterface` (optional, but recommended)

`@FunctionalInterface`

```
interface MyFunctional {  
  
    void execute();  
  
}
```

✓ Examples:

- Runnable, Callable, Comparator
 - Java 8 Built-ins: Function, Predicate, Consumer, Supplier
-

◆ 3. Predicate

Used for boolean conditions (true/false checks)

```
Predicate<String> isLong = str -> str.length() > 5;
```

```
System.out.println(isLong.test("hello")); // false
```

✓ Use Case:

- Filter collections

```
List<String> names = List.of("John", "Elizabeth", "Tom");
```

```
names.stream().filter(name -> name.length() > 4).forEach(System.out::println);
```

◆ 4. Function<T, R>

Takes input T, returns output R

```
Function<String, Integer> strLength = str -> str.length();
```

```
System.out.println(strLength.apply("Hello")); // 5
```

✅ **Use Case:**

- Data transformation

```
List<String> names = List.of("apple", "banana");
```

```
List<Integer> lengths = names.stream().map(str -> str.length()).collect(Collectors.toList());
```

◆ **5. Consumer**

Takes input, returns nothing

```
Consumer<String> printUpper = str -> System.out.println(str.toUpperCase());
```

```
printUpper.accept("hello"); // HELLO
```

✅ **Use Case:**

- Logging, printing, processing without returning
-

◆ **6. Supplier**

Takes no input, returns something

```
Supplier<Double> randomSupplier = () -> Math.random();
```

```
System.out.println(randomSupplier.get());
```

✅ **Use Case:**

- Random data generation, lazy initialization
-

◆ **7. Default Methods in Interface**

- Methods with implementation in interface (Java 8+)
- Does not break old implementations

```
interface Vehicle {  
    default void start() {  
        System.out.println("Vehicle started");  
    }  
}
```

◆ 8. Static Methods in Interface

- Belongs to the interface, not an instance
- Can't be overridden

```
interface Utility {  
    static void log(String msg) {  
        System.out.println("Log: " + msg);  
    }  
}  
  
Utility.log("Test");
```

◆ 1. BiPredicate<T, U>

Represents a function that takes **two inputs** and returns a **boolean** result.

✓ Syntax:

```
BiPredicate<T, U> condition = (t, u) -> booleanExpression;
```

✓ Example:

```
BiPredicate<String, String> areEqual = (s1, s2) -> s1.equalsIgnoreCase(s2);  
System.out.println(areEqual.test("Java", "java")); // true
```

✓ Use Case:

- Comparing two values
 - Filtering based on two fields (e.g., user and role)
-

◆ 2. BiFunction<T, U, R>

Represents a function that takes **two inputs** and returns a **result**.

✓ Syntax:

```
BiFunction<T, U, R> function = (t, u) -> returnValue;
```

✓ Example:

```
BiFunction<Integer, Integer, String> sumLabel = (a, b) -> "Sum: " + (a + b);
```

```
System.out.println(sumLabel.apply(3, 7)); // "Sum: 10"
```

✓ Use Case:

- Processing or mapping two inputs into a result (e.g., fullName from first + last)
-

◆ 3. BiConsumer<T, U>

Represents an operation that takes **two inputs** and **returns nothing**.

✓ Syntax:

```
BiConsumer<T, U> action = (t, u) -> { // side effect };
```

✓ Example:

```
BiConsumer<String, Integer> printKV = (k, v) -> System.out.println(k + " -> " + v);
```

```
printKV.accept("Age", 30); // "Age -> 30"
```

✓ Use Case:

- Logging key-value pairs
- Populating or iterating over maps

◆ 1. UnaryOperator<T>

✔ Definition:

A function that **takes one argument** and **returns a result of the same type**.

@FunctionalInterface

```
public interface UnaryOperator<T> extends Function<T, T> { }
```

✔ Example:

```
UnaryOperator<String> toUpper = str -> str.toUpperCase();
```

```
System.out.println(toUpper.apply("java")); // Output: JAVA
```

✔ Use Cases:

- Data transformation (e.g., string trimming, math operations)
- Replacing Function<T, T> where input and output are same type

```
List<String> names = List.of(" alice ", "bob ");
```

```
List<String> cleaned = names.stream().map(String::trim).toList();
```

◆ 2. BinaryOperator<T>

✔ Definition:

A function that **takes two arguments** and **returns a result of the same type**.

@FunctionalInterface

```
public interface BinaryOperator<T> extends BiFunction<T, T, T> { }
```

✔ Example:

```
BinaryOperator<Integer> add = (a, b) -> a + b;
```

```
System.out.println(add.apply(10, 20)); // Output: 30
```

✔ Use Cases:

- Aggregations (sum, min, max)
- Merging data
- Replacing BiFunction<T, T, T>

◆ Method References in Java 8

Method reference is a shorthand for a **lambda expression** that simply calls a method.

✓ Syntax:

```
ClassName::methodName
```

✓ Example 1: Print each element (using `System.out::println`)

```
List<String> students = Arrays.asList("Ram", "Shyam", "Ghanshyam");
```

```
// Using Lambda:
```

```
students.forEach(x -> System.out.println(x));
```

```
// Using Method Reference:
```

```
students.forEach(System.out::println);
```

✓ Benefit: Cleaner and more readable when all you're doing is calling a method.

◆ Constructor Reference

Used to **create objects** via method reference in place of lambdas like `x -> new ClassName(x)`.

✓ Syntax:

```
ClassName::new
```

✓ Example 2: Constructor reference with `.map()` in a stream

```
class MobilePhone {
```

```
    String name;
```

```
    MobilePhone(String name) {
```

```
        this.name = name;
```

```

    }
}

List<String> names = Arrays.asList("A", "B", "C");

```

// Using Lambda:

```

List<MobilePhone> list1 = names.stream()
    .map(name -> new MobilePhone(name))
    .collect(Collectors.toList());

```

// Using Constructor Reference:

```

List<MobilePhone> list2 = names.stream()
    .map(MobilePhone::new)
    .collect(Collectors.toList());

```

Types of Method References

Type	Example	Equivalent Lambda
Static Method	ClassName::staticMethod	x -> ClassName.staticMethod(x)
Instance Method (specific object)	object::instanceMethod	x -> object.instanceMethod(x)
Instance Method (any object of class)	ClassName::instanceMethod	(x, y) -> x.instanceMethod(y)
Constructor	ClassName::new	x -> new ClassName(x)

Java Stream API –

📌 1. Introduction to Streams

◆ What is a Stream?

- A **Stream** is a sequence of elements that supports **functional-style operations** on collections of elements.
- Introduced in **Java 8** as part of the `java.util.stream` package.

◆ Why Use Streams?

- **Simplify Data Processing**
- **Embrace Functional Programming**
- **Improve Readability and Maintainability**
- **Enable Easy Parallelism** with `.parallelStream()`

📌 2. Components of a Stream

A stream operation typically involves **three parts**:

Component	Description
Source	The data source: Collection, Array, etc.
Intermediate Ops	Operations like filter, map, sorted – these are lazy and chainable
Terminal Operation	Operations like count, collect, forEach – triggers the stream pipeline

📌 3. Code Walkthrough – Example

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
```

```
System.out.println(numbers.stream().filter(x -> x % 2 == 0).count());
```

🔍 Step-by-Step Breakdown:

1. `numbers` is a `List`: `[1, 2, 3, 4, 5]`
2. `.stream()` converts the list into a stream

3. `.filter(x -> x % 2 == 0)` keeps only even numbers → [2, 4]
4. `.count()` returns how many elements are left → 2
5. `System.out.println()` prints the result: 2

💡 This demonstrates filtering and terminal operation in a concise, functional style.

📌 4. Creating Streams

✅ 1. From Collections

```
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
```

```
Stream<Integer> stream = list.stream();
```

✅ 2. From Arrays

```
String[] array = {"a", "b", "c"};
```

```
Stream<String> stream1 = Arrays.stream(array);
```

✅ 3. Using `Stream.of()`

```
Stream<String> stream2 = Stream.of("a", "b");
```

✅ 4. Infinite Streams

```
Stream<Integer> infinite1 = Stream.generate(() -> 1); // Constant stream
```

```
Stream<Integer> infinite2 = Stream.iterate(1, x -> x + 1); // Increasing numbers
```

💡 Use `.limit(n)` with infinite streams to avoid infinite loops.

✅ Java Stream API – Intermediate Operations

◆ What are Intermediate Operations?

- **Intermediate operations** transform one stream into another.
 - They are **lazy** – they don't execute until a **terminal operation** is invoked (like `count()`, `collect()`).
-

List of Common Intermediate Operations

Operation Description

<code>filter()</code>	Filters elements based on a condition
<code>map()</code>	Transforms each element (e.g., changing case, squaring, etc.)
<code>sorted()</code>	Sorts the elements (natural order or custom comparator)
<code>distinct()</code>	Removes duplicate elements
<code>limit()</code>	Limits the number of elements in the stream
<code>skip()</code>	Skips the first N elements
<code>peek()</code>	Performs side-effects for debugging/logging
<code>flatMap()</code>	Flattens nested collections or splits elements

Examples with Explanation

1. `filter()` – Filter elements based on a predicate

```
List<String> list = Arrays.asList("Akshit", "Ram", "Shyam", "Ghanshyam", "Akshit");
```

```
Stream<String> filteredStream = list.stream().filter(x -> x.startsWith("A"));
```

- **Lazy:** No filtering happens until a terminal op like `count()`:

```
long res = list.stream().filter(x -> x.startsWith("A")).count();
```

```
System.out.println(res); // Output: 2
```

2. `map()` – Transform each element

```
Stream<String> stringStream = list.stream().map(String::toUpperCase);
```

- Transforms: "Akshit" → "AKSHIT", etc.
-

3. `sorted()` – Sort elements

```
Stream<String> sortedStream = list.stream().sorted(); // Natural order
```

```
Stream<String> sortedByLength = list.stream().sorted((a, b) -> a.length() - b.length()); //
Custom sort
```

4. distinct() – Remove duplicates

```
System.out.println(list.stream().filter(x -> x.startsWith("A")).distinct().count()); // Output: 1
```

5. limit(n) – Restrict the number of elements

```
System.out.println(Stream.iterate(1, x -> x + 1).limit(100).count()); // Output: 100
```

6. skip(n) – Skip first N elements

```
System.out.println(Stream.iterate(1, x -> x + 1).skip(10).limit(100).count()); // Output: 100
```

7. peek() – Debug or perform side-effects

```
Stream.iterate(1, x -> x + 1)
    .skip(10)
    .limit(100)
    .peek(System.out::println) // Prints each element
    .count();
```

🧠 Use only for debugging; don't mutate elements here.

8. flatMap() – Flatten and transform nested data

Flatten a list of lists

```
List<List<String>> listOfLists = Arrays.asList(
    Arrays.asList("apple", "banana"),
    Arrays.asList("orange", "kiwi"),
    Arrays.asList("pear", "grape")
)
```

```
);
```

```
List<String> flatList = listOfLists.stream()  
    .flatMap(x -> x.stream())  
    .map(String::toUpperCase)  
    .toList();  
System.out.println(flatList);
```

Flatten sentences into words

```
List<String> sentences = Arrays.asList(  
    "Hello world",  
    "Java streams are powerful",  
    "flatMap is useful"  
);
```

```
List<String> words = sentences.stream()  
    .flatMap(sentence -> Arrays.stream(sentence.split(" ")))  
    .map(String::toUpperCase)  
    .toList();  
System.out.println(words);
```

Java Stream API – Terminal Operations

◆ What are Terminal Operations?

- They trigger the **execution** of the stream pipeline.
 - They produce a **non-stream result**: a primitive, a collection, or no value (e.g., printing).
-

List of Terminal Operations with Examples

Operation	Description	Returns
collect()	Converts stream to collection or string	List, Set, etc.
forEach()	Iterates through each element (order not guaranteed)	void
reduce()	Combines elements to a single result	Optional
count()	Counts number of elements	long
anyMatch()	Checks if any match a predicate	boolean
allMatch()	Checks if all match a predicate	boolean
noneMatch()	Checks if none match a predicate	boolean
findFirst()	Returns first element	Optional
findAny()	Returns any one element (parallel-friendly)	Optional
toArray()	Converts to array	Object[]
min() / max()	Finds min/max using comparator	Optional
forEachOrdered()	Processes in encounter order (parallel-safe)	void

Examples with Explanation

1. collect() – Collect results into a collection

```
list.stream().skip(1).collect(Collectors.toList());  
list.stream().skip(1).toList(); // Java 16+
```

2. forEach() – Terminal iteration

```
list.stream().forEach(x -> System.out.println(x));
```

3. reduce() – Combines elements into a single result

```
Optional<Integer> sum = list.stream().reduce(Integer::sum);  
System.out.println(sum.get()); // Output: 6
```

4. count() – Count elements

```
System.out.println(list.stream().count()); // Output: 3
```

Match Operations

5. anyMatch(), allMatch(), noneMatch()

```
System.out.println(list.stream().anyMatch(x -> x % 2 == 0)); // true
```

```
System.out.println(list.stream().allMatch(x -> x > 0)); // true
```

```
System.out.println(list.stream().noneMatch(x -> x < 0)); // true
```

Finding Elements

6. findFirst() / findAny()

```
System.out.println(list.stream().findFirst().get()); // 1
```

```
System.out.println(list.stream().findAny().get()); // 1 (not guaranteed in parallel)
```

Other Useful Terminal Operations

7. toArray()

```
Object[] array = Stream.of(1, 2, 3).toArray();
```

8. min() / max()

```
System.out.println(Stream.of(2, 44, 69).max((a, b) -> a - b).get()); // max
```

```
System.out.println(Stream.of(2, 44, 69).min(Comparator.naturalOrder()).get()); // min
```

9. forEachOrdered() – Order-preserving for parallel streams

```
numbers0.parallelStream().forEach(System.out::println); // May be out of order
```

```
numbers0.parallelStream().forEachOrdered(System.out::println); // In order
```

Practical Examples

✓ Filtering and Collecting

```
List<String> names = Arrays.asList("Anna", "Bob", "Charlie", "David");  
System.out.println(names.stream().filter(x -> x.length() > 3).toList());
```

✓ Squaring and Sorting Numbers

```
List<Integer> numbers = Arrays.asList(5, 2, 9, 1, 6);  
System.out.println(numbers.stream().map(x -> x * x).sorted().toList());
```

✓ Summing Values

```
List<Integer> integers = Arrays.asList(1, 2, 3, 4, 5);  
System.out.println(integers.stream().reduce(Integer::sum).get()); // 15
```

✓ Counting Occurrences of a Character

```
String sentence = "Hello world";  
System.out.println(sentence.chars().filter(x -> x == 'l').count()); // 3
```

! Important Notes

- **Streams can't be reused** once a terminal operation is called:

```
Stream<String> stream = names.stream();  
stream.forEach(System.out::println);  
  
// stream.map(...).toList(); ✗ IllegalStateException
```

- **Stateful vs Stateless:**

- filter, map = stateless
- sorted, distinct, limit = stateful

Here's a **clear, structured explanation** of your code on **parallel streams** and **stream performance** with practical notes and key takeaways:

Java Stream Performance: Sequential vs Parallel Streams

◆ What is a Parallel Stream?

- A **parallel stream** divides the workload across multiple threads.
 - Each thread processes a portion of the stream **simultaneously**.
 - It can improve performance significantly on **large, CPU-intensive datasets**.
-

Code Breakdown

✓ Step 1: Generate List of 20,000 Integers

```
List<Integer> list = Stream.iterate(1, x -> x + 1).limit(20000).toList();
```

✓ Step 2: Time the Sequential Stream Execution

```
List<Long> factorialsList = list.stream().map(ParallelStream::factorial).toList();
```

- Each number's factorial is calculated one-by-one (single thread).
- Timing:

```
System.out.println("Time taken with sequential stream: " + (endTime - startTime) + " ms");
```

✓ Step 3: Time the Parallel Stream Execution

```
factorialsList = list.parallelStream().map(ParallelStream::factorial).toList();
```

- The task is split across threads.
- Ideal for CPU-bound operations like factorial.

```
System.out.println("Time taken with parallel stream: " + (endTime - startTime) + " ms");
```

Notes on Performance

- **Effective for:** Heavy CPU operations (e.g., factorial, image processing).
 - **Not useful for:** I/O bound, small, or simple operations due to thread management overhead.
 - Order-sensitive tasks (like cumulative sum) should not use parallel streams.
-

Sequential vs Parallel Performance: Example Output

(Sample results vary by machine)

Time taken with sequential stream: 190 ms

Time taken with parallel stream: 55 ms

Cumulative Sum Example

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
```

```
AtomicInteger sum = new AtomicInteger(0);
```

```
List<Integer> cumulativeSum =  
numbers.stream().sequential().map(sum::addAndGet).toList();
```

- **Expected:** [1, 3, 6, 10, 15]
 - **Actual with parallel stream:**  Unpredictable due to non-deterministic thread execution
 - Use `.sequential()` for such **stateful operations**
-

Key Takeaways

Good Use Cases

Heavy computations (e.g., factorial)

Large collections

Independent, stateless operations

Bad Use Cases

Order-sensitive tasks (e.g., cumulative sum)

Small datasets

Stateful operations (e.g., `addAndGet`)

Great set of examples! Here's a clean **annotated summary** and **explanation** of your usage of the `Collectors` utility class in Java Streams, organized for clarity and learning.

Java Collectors Summary Guide

✓ 1. Collecting to a List

```
List<String> res = names.stream()  
    .filter(name -> name.startsWith("A"))  
    .collect(Collectors.toList());
```

- ◆ **Purpose:** Collect filtered items into a List.
-

✓ 2. Collecting to a Set

```
Set<Integer> set = nums.stream().collect(Collectors.toSet());
```

- ◆ **Removes duplicates** by collecting into a Set.
-

✓ 3. Collecting to a Specific Collection

```
ArrayDeque<String> collect = names.stream()  
    .collect(Collectors.toCollection(() -> new ArrayDeque<>()));
```

- ◆ You can define the **target collection type** explicitly.
-

✓ 4. Joining Strings

```
String concatenatedNames = names.stream()  
    .map(String::toUpperCase)  
    .collect(Collectors.joining(", "));
```

- ◆ Joins elements into a **single string**, with optional delimiter.
-

✓ 5. Summarizing Data

```
IntSummaryStatistics stats = numbers.stream()  
    .collect(Collectors.summarizingInt(x -> x));
```

◆ Gives you:

- Count
 - Sum
 - Min
 - Max
 - Average
-

✓ 6. Calculating Averages

```
Double average = numbers.stream()
    .collect(Collectors.averagingInt(x -> x));
```

◆ One-liner to compute the **average** of numeric values.

✓ 7. Counting Elements

```
Long count = numbers.stream().collect(Collectors.counting());
```

◆ Shorthand for `.count()`, useful in pipelines.

✓ 8. Grouping Elements

```
words.stream().collect(Collectors.groupingBy(String::length));
```

```
words.stream().collect(Collectors.groupingBy(String::length, Collectors.joining(", ")));
```

```
words.stream().collect(Collectors.groupingBy(String::length, Collectors.counting()));
```

```
TreeMap<Integer, Long> treeMap = words.stream().collect(
    Collectors.groupingBy(String::length, TreeMap::new, Collectors.counting()));
```

- ◆ Groups elements by a classifier (`String::length`)
 - ◆ Custom downstream collectors (e.g., `joining`, `counting`)
 - ◆ Custom map type (`TreeMap`)
-

✓ 9. Partitioning Elements

```
words.stream().collect(Collectors.partitioningBy(x -> x.length() > 5));
```

- ◆ Splits into **2 groups**: true and false based on predicate.
-

✓ 10. Mapping Before Collecting

```
words.stream().collect(Collectors.mapping(String::toUpperCase, Collectors.toList()));
```

- ◆ Apply transformation before collecting.
-

✓ 11. Advanced Examples

📌 Example 1: Group by Length

```
l1.stream().collect(Collectors.groupingBy(String::length));
```

📌 Example 2: Word Occurrence Counter

```
Arrays.stream(sentence.split(" "))  
    .collect(Collectors.groupingBy(x -> x, Collectors.counting()));
```

- ◆ **Frequency counter.**

📌 Example 3: Even/Odd Partition

```
l2.stream().collect(Collectors.partitioningBy(x -> x % 2 == 0));
```

📌 Example 4: Sum of Map Values

```
items.values().stream().reduce(Integer::sum);  
items.values().stream().collect(Collectors.summingInt(x -> x));
```

- ◆ Both calculate the total value.

📌 Example 5: Stream → Map

```
fruits.stream().collect(Collectors.toMap(  
    x -> x.toUpperCase(),  
    x -> x.length()));
```


- ◆ Keys = Uppercase name, Values = Length

✚ Example 6: Frequency Map

```
words2.stream().collect(Collectors.toMap(  
    k -> k, v -> 1, Integer::sum));
```

- ◆ Uses a **merge function** to handle duplicates.
-

Primitive Streams in Java

1. Creating Primitive Streams

- From an array:

```
int[] numbers = {1, 2, 3, 4, 5};
```

```
IntStream stream = Arrays.stream(numbers);
```

- From a range of integers:

```
IntStream.range(1, 5)    // 1, 2, 3, 4 (exclusive upper bound)
```

```
IntStream.rangeClosed(1, 5) // 1, 2, 3, 4, 5 (inclusive upper bound)
```

You can box the primitive values into wrapper objects with `.boxed()`:

```
List<Integer> list = IntStream.range(1, 5)
```

```
    .boxed()
```

```
    .collect(Collectors.toList());
```

```
System.out.println(list); // Output: [1, 2, 3, 4]
```

2. Creating from literals

```
IntStream.of(1, 2, 3);
```

3. DoubleStream from Random

```
DoubleStream doubles = new Random().doubles(5);
```

- This creates a stream of 5 random double values.

- You can collect them into a list of Double using `.boxed().toList()`.
- Example:

```
System.out.println(doubles.boxed().toList());
```

4. Common DoubleStream operations (commented out but useful):

```
// doubles.sum();           // sum of elements
// doubles.min();           // OptionalDouble with min value
// doubles.max();           // OptionalDouble with max value
// doubles.average();       // OptionalDouble with average
// doubles.summaryStatistics(); // DoubleSummaryStatistics object
// doubles.mapToInt(x -> (int)(x + 1)); // convert to IntStream with mapping
```

5. IntStream from Random

```
IntStream intStream = new Random().ints(5);
System.out.println(intStream.boxed().toList());
```

Java Iterator in Collections

What is an Iterator?

- An **Iterator** is an object that enables traversing (iterating) over elements in a collection (like List, Set, etc.).
 - It provides a way to loop through a collection **without exposing the underlying structure**.
 - Mainly used to:
 - Access elements sequentially.
 - Remove elements safely while iterating.
-

Basic methods of Iterator interface:

Method	Description
boolean hasNext()	Returns true if there is a next element.
E next()	Returns the next element. Throws NoSuchElementException if none left.
void remove()	Removes the last element returned by next(). Optional operation.

Example usage:

```
List<String> names = new ArrayList<>(Arrays.asList("Alice", "Bob", "Charlie"));
```

```
// Getting an iterator from the collection
```

```
Iterator<String> iterator = names.iterator();
```

```
// Iterating over elements
```

```
while (iterator.hasNext()) {
```

```
    String name = iterator.next();
```

```
    System.out.println(name);
```

```
    if (name.equals("Bob")) {
```

```
        // Safe to remove element during iteration
```

```
        iterator.remove();
```

```
    }
```

```
}
```

```
System.out.println("After removal: " + names);
```

Output:

Alice

Bob

Charlie

After removal: [Alice, Charlie]

Why use Iterator?

- **Safe removal:** You can remove elements from a collection safely during iteration without `ConcurrentModificationException`.
 - **Uniform access:** Works across any collection implementing `Iterable`.
 - **Legacy support:** Before Java 8 streams, `Iterator` was the primary way to loop.
-

Iterator vs For-each loop:

- For-each loop is simpler and recommended if you only need to read elements.
 - Use `Iterator` when:
 - You need to remove elements during traversal.
 - You want explicit control over iteration.
-